RAG vs. GraphRAG: A Systematic Evaluation and Key Insights

Haoyu Han
Michigan State University
East Lansing, MI, USA

Li Ma*
Michigan State University
East Lansing, MI, USA

Yu Wang
University of Oregon
Eugene, OR, USA

Harry Shomer
University of Texas at Arlington
Arlington, TX, USA

Kai Guo
Michigan State University
East Lansing, MI, USA

Yongjia Lei
University of Oregon
Eugene, OR, USA

Zhisheng Qi
University of Oregon
Eugene, OR, USA

Zhigang Hua
Meta Platforms, Inc.
Menlo Park, CA, USA

Bo Long
Meta Platforms, Inc.
Menlo Park, CA, USA

Hui Liu
Michigan State University
East Lansing, MI, USA

Charu Aggarwal
IBM T.J. Watson Research Center
Yorktown Heights, NY, USA

Jiliang Tang
Michigan State University
East Lansing, MI, USA

Abstract

Retrieval-Augmented Generation (RAG) improves large language models (LLMs) by retrieving relevant information from external sources and has been widely adopted for text-based tasks. For structured data, such as knowledge graphs, Graph Retrieval-Augmented Generation (GraphRAG) retrieves and aggregates information along graph structures. More recently, GraphRAG has been extended to general text settings by organizing unstructured text into graph representations, showing promise for reasoning and grounding. Despite these advances, existing GraphRAG systems for text data are often tailored to specific tasks, datasets, and system designs, resulting in heterogeneous evaluation protocols. Consequently, a systematic understanding of the relative strengths, limitations, and trade-offs between RAG and GraphRAG on widely used text benchmarks remains limited. In this paper, we present a comprehensive benchmark study comparing RAG and GraphRAG on established text-based tasks, including question answering and query-based summarization. We introduce a unified evaluation protocol that standardizes data preprocessing, retrieval configurations, and generation settings, enabling fair and reproducible comparisons. Our results highlight the distinct strengths of RAG and GraphRAG across different tasks and evaluation perspectives. Building on these findings, we explore selection and integration strategies that combine the strengths of both paradigms, leading to consistent performance improvements. We further analyze failure modes, efficiency trade-offs, and evaluation biases, and highlight key considerations for designing and evaluating retrieval-augmented generation systems. Our code is available at https://github.com/haoyuhan1/RAGvsGraphRAG.

*Corresponding Authors.

KDD '26, Jeju Island, Republic of Korea

ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817575

CCS Concepts

• **Information systems** → **Retrieval effectiveness**; **Question answering**; **Summarization**.

Keywords

Retrieval-Augmented Generation; GraphRAG; Large Language Models; Question Answering; Summarization

ACM Reference Format:
Haoyu Han, Li Ma, Yu Wang, Harry Shomer, Kai Guo, Yongjia Lei, Zhisheng Qi, Zhigang Hua, Bo Long, Hui Liu, Charu Aggarwal, and Jiliang Tang. 2026. RAG vs. GraphRAG: A Systematic Evaluation and Key Insights. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770855.3817575

1 Introduction

Retrieval-Augmented Generation (RAG) has emerged as a practical paradigm for improving downstream tasks by retrieving relevant knowledge from external data sources. It has been successfully deployed in a wide range of real-world applications, including healthcare [48], law [42], finance [54], and education [29]. With the advent of Large Language Models (LLMs), integrating retrieval into generation further improves faithfulness by mitigating hallucinations and enhancing robustness [14, 56]. In most existing RAG systems, retrieval is performed over text corpora.

Graphs provide an explicit representation of relational structure and have long been used across domains such as knowledge representation, social networks, and biomedical discovery [27, 43, 44]. Graph Retrieval-Augmented Generation (GraphRAG) has recently gained attention for retrieving and aggregating information from graph-structured data, including knowledge graphs (KGs) and molecular graphs [12, 31]. Beyond leveraging existing graphs, an emerging line of work extends GraphRAG to text-based tasks by constructing graphs from unstructured documents, with reported benefits for global summarization [6], planning [24], and reasoning [13]. However, most GraphRAG studies for text are conducted under task- and system-specific settings, using bespoke datasets, graph construction heuristics, and evaluation protocols. This heterogeneity makes it difficult to draw principled conclusions about *when and why* explicit graph structures help (or hurt) retrieval-augmented generation, and obscures practical trade-offs such as construction cost, retrieval latency, and storage footprint.

To address this gap, we conduct a controlled and systematic benchmark of RAG and GraphRAG on widely used text-based tasks, focusing on *Question Answering* (QA) and *Query-based Summarization.* We consider four representative categories of GraphRAG systems: **(1)** *KG-based GraphRAG* [25], which extracts a KG from text and performs retrieval over the KG; **(2)** *Community-based GraphRAG* [6], which performs retrieval over community structures and hierarchical abstractions; **(3)** *Text-centric graph-guided RAG* [17], which retrieves original text chunks with the assistance of a constructed knowledge graph; and **(4)** *Hierarchical summary-based GraphRAG* [33], which builds hierarchical summaries to enable multi-granular retrieval without relying on explicit KGs. For QA, we evaluate both single-hop and multi-hop settings; for summarization, we consider both single-document and multi-document scenarios. Crucially, we introduce a unified evaluation protocol that standardizes data preprocessing, retrieval, and generation settings, enabling fair and reproducible comparisons across paradigms.

Our analysis results lead to several key findings. First, **RAG and GraphRAG exhibit complementary behaviors rather than a consistent winner**. In QA, RAG performs better on single-hop and detail-oriented factual queries, whereas GraphRAG is more effective on multi-hop, reasoning-intensive questions. Second, **GraphRAG design choices matter**: for example, community-based global search can sacrifice query-specific details, hurting detail-oriented QA, while providing more corpus-level aggregation that benefits broad or diverse summarization outputs. Third, **evaluation protocol can change conclusions**: we show that LLM-as-a-Judge evaluation for summarization can be highly sensitive to the presentation order of candidate summaries, introducing strong position effects that may confound comparisons. Finally, **GraphRAG is not free**: it often incurs higher construction cost, retrieval latency, and storage footprint, and its performance can be sensitive to the quality (and cost) of graph construction. These findings suggest that effective retrieval-augmented generation should not treat RAG and GraphRAG as mutually exclusive choices. Motivated by this, we study two practical hybrid strategies: **Selection**, which routes queries to RAG or GraphRAG based on query type for efficiency, and **Integration**, which combines evidence from both paradigms to maximize performance. Across benchmarks, these strategies yield consistent improvements. Our main contributions are as follows:

- **Systematic Benchmark:** We present a controlled benchmark comparing RAG and multiple GraphRAG variants across QA and query-based summarization under a unified evaluation protocol (consistent preprocessing, retrieval, and generation settings) for fair and reproducible comparison.
- **Strong, Task-Level Findings:** We identify clear complementarities: RAG is stronger for factual/detail-oriented QA, while GraphRAG benefits reasoning-intensive QA and produces more corpus-level, diverse summaries, with outcomes strongly affected by GraphRAG design choices (e.g., local vs. global search).
- **Hybrid Strategies:** We study **Selection** and **Integration** strategies that combine RAG and GraphRAG, achieving consistent improvements and illustrating effectiveness–efficiency trade-offs.
- **Evaluation and Efficiency Analyses:** We analyze failure modes, construction/retrieval/storage costs, sensitivity to graph construction quality, and demonstrate strong position effects in LLM-as-a-Judge summarization evaluation, highlighting practical considerations for reliable (Graph)RAG assessment.

2 Related Works

Retrieval-Augmented Generation (RAG) has been widely used to enhance LLMs by retrieving relevant information from external sources [8, 10]. Most RAG systems process text corpora by splitting documents into passages or chunks [9]. Given a query, relevant chunks are retrieved using lexical search [32], dense semantic retrieval [18], or hybrid retrieval strategies. Beyond vanilla retrieval, prior work has explored pre-retrieval processing, such as query rewriting and query expansion [26, 57]; post-retrieval processing, such as reranking, filtering, and compression [3, 47]; and fine-tuning strategies to better align retrievers or generators with downstream tasks [21]. These techniques have improved RAG across a wide range of applications, including question answering [49], dialogue generation [15], and summarization [16]. Recent studies also benchmark RAG pipelines and evaluation tools across tasks, domains, and metrics [2, 7, 52].

While standard RAG typically treats documents as independent text chunks, many real-world information sources contain rich relational structures, such as knowledge graphs, social graphs, citation networks, and molecular graphs [27, 45]. Graph Retrieval-Augmented Generation (GraphRAG) incorporates such graph structures into retrieval and generation to exploit relational signals among connected entities, documents, or chunks [12, 31]. Early work mainly focuses on retrieval over existing knowledge graphs for downstream tasks such as knowledge-graph question answering [36, 51] and fact checking [19]. Graph structures have also been used to improve text-centric retrieval; for example, hyperlink or document graphs can guide evidence selection for question answering [22]. More recently, GraphRAG methods have begun to construct graphs directly from text corpora to support general text-based tasks [12]. One line of work builds document- or chunk-level graphs to guide retrieval over textual units [3, 22]. Another line constructs entity–relation graphs, often with LLM assistance, and retrieves information at multiple abstraction levels, such as local entity neighborhoods or global community-level summaries [6, 13]. Related hierarchical approaches use graph-inspired structures without requiring fully specified entity–relation semantics. For example, RAPTOR constructs hierarchical summary trees for multi-granular retrieval [33], while HippoRAG and its extensions build entity-linked memory graphs to guide chunk retrieval [11, 17].

Despite rapid progress, GraphRAG systems for text are often evaluated under heterogeneous protocols, with varying graph construction methods, retrieval configurations, and even evaluation criteria. Moreover, graph construction introduces additional costs (indexing time, retrieval latency, and storage footprint) and can be sensitive to the quality of the construction model, yet these trade-offs are not consistently characterized across studies. As a result, it remains unclear how GraphRAG compares with standard RAG on general text-based benchmarks and what practical trade-offs it entails. This motivates our systematic benchmark evaluation under unified experimental settings.

3 Evaluation Framework

In this section, we describe our evaluation framework and experimental protocol. To ensure fair comparison, we evaluate RAG and GraphRAG under identical settings whenever applicable, and otherwise follow the default configurations of each method while matching key budgets. We decouple retrieval from generation by first saving retrieved evidence for each method and then using a unified generation script to produce outputs conditioned on the saved retrieval results.

3.1 RAG Pipeline

We adopt a standard dense-retrieval RAG pipeline [18]. Given a corpus, we segment documents into textual chunks and build an index by embedding each chunk into a shared vector space. At inference time, we embed the query, retrieve top-ranked chunks based on the semantic similarity.

3.2 GraphRAG Implementations

GraphRAG designs differ in how structures are constructed and how structural information is used during retrieval. In this work, we group GraphRAG approaches into four representative classes and select representative implementations for each class:

KG-based GraphRAG. In KG-based GraphRAG [25], a knowledge graph is constructed from text. Given a query, relevant entities are identified and aligned to nodes in the KG, and retrieval traverses multi-hop neighborhoods to collect relational triplets *(head, relation, tail)* as evidence. We consider two variants: **(1)** *KG-GraphRAG (Triplets)*, which retrieves only triplets, and **(2)** *KG-GraphRAG (Triplets+Text)*, which retrieves both triplets and their associated source text. We implement KG-GraphRAG using LlamaIndex [25].[1]

Community-based GraphRAG. Community-based GraphRAG [6] further organizes the constructed KG into hierarchical communities using graph clustering algorithms. Each community is associated with a textual summary/report, where lower-level communities capture fine-grained information and higher-level communities provide increasingly abstract representations. We evaluate two retrieval modes: **Local Search** retrieves entity neighborhoods and lower-level community reports via entity matching, denoted as *Community-GraphRAG (Local)*; **Global Search** retrieves high-level community summaries by semantic similarity, denoted as *Community-GraphRAG (Global)*. We adopt the implementation of Edge et al. [6].[2]

Text-centric graph-guided RAG. Text-centric graph-guided methods retain original text chunks as the primary retrieval units while leveraging graph structures to guide scoring or traversal. We select HippoRAG2 [11] as a representative method. HippoRAG2 builds an entity-linked graph over chunks and retrieves query-relevant entities first, followed by the connected text chunks. Here, the graph acts as an auxiliary structure guiding chunk retrieval rather than the primary retrieval target.

Hierarchical summary-based GraphRAG. Hierarchical summary-based methods construct multi-level hierarchical structures over text, where higher-level nodes represent progressively more abstract summaries of lower-level content. We adopt RAPTOR [33] as a representative method. RAPTOR recursively clusters text chunks and generates summaries at each level, enabling coarse-to-fine, multi-granular retrieval without relying on explicit KGs.

3.3 Tasks

We evaluate all methods on two representative text-based tasks: Question Answering and Query-based Summarization, covering both single-hop and multi-hop QA, as well as single-document and multi-document summarization. For single-document tasks, retrieval is restricted to the corresponding document, while for multi-document tasks, retrieval is performed over an index constructed from all documents.

3.4 Unified Experimental Settings

To ensure fair comparison across RAG and GraphRAG methods, we standardize core settings whenever applicable.

Graph construction. For GraphRAG methods that require graph construction from text (e.g., KG-GraphRAG, Community-GraphRAG, and HippoRAG2), graphs are constructed using GPT-4o-mini; results with GPT-4o are reported in Appendix B.

Chunking. We segment documents into chunks of approximately *256* tokens for all methods.

Embedding model and retrieval budget. We embed queries, chunks, and graph information into a shared vector space using OpenAI's `text-embedding-ada-002` model [30], and retrieve top-k candidates by semantic similarity, where k=10 by default.

Reranking. When reranking is enabled, we first retrieve the top $2k$ candidates and apply a cross-encoder reranker to reorder retrieved candidates and select the final top-k evidence units. We use `BAAI/bge-reranker-large` [46] as the reranker for all methods that support reranking, ensuring consistent reranking behavior across systems.

Iterative retrieval. When iterative retrieval is enabled, we adopt IRCoT [37], which interleaves retrieval with intermediate reasoning steps.

Generation backbones. To control for generation capacity, we use two open-source instruction-tuned LLMs of different sizes as generators: Llama-3.1-8B-Instruct and Llama-3.1-70B-Instruct [5].

4 Question Answering

Question Answering (QA) is one of the most widely used tasks for evaluating the performance of RAG systems. To systematically assess the effectiveness of RAG and GraphRAG on general QA settings, we evaluate representative methods on widely used datasets and follow standard metrics used in prior work.

4.1 Datasets and Evaluation Metrics

To comprehensively evaluate the performance of GraphRAG on general QA tasks, we select four widely used datasets that cover different perspectives. For the single-hop QA task, we select the

[1] https://www.llamaindex.ai/
[2] https://microsoft.github.io/graphrag

Natural Questions (NQ) dataset [20]. For the multi-hop QA task, we select HotPotQA [50] and MultiHop-RAG [35] datasets. The MultiHop-RAG dataset categorizes queries into four types: Inference, Comparison, Temporal, and Null queries. To further analyze the performance of RAG and GraphRAG at a finer granularity, we also include NovelQA [39], which contains 21 different types of queries. We use Precision (P), Recall (R), and F1-score as evaluation metrics for the NQ and HotPotQA datasets, while accuracy is used for the MultiHop-RAG and NovelQA datasets following their original papers.

Table 1: Performance comparison (%) on NQ and Hotpot.

Method	NQ			Hotpot		
	P	R	F1	P	R	F1
RAG	**71.70**	**63.93**	**64.78**	62.32	60.47	60.04
RaptorRAG	66.06	59.56	60.04	63.81	61.46	61.31
KG-GraphRAG (Triplets only)	40.09	33.56	34.28	26.88	24.81	25.02
KG-GraphRAG (Triplets+Text)	58.36	48.93	50.27	45.22	42.85	42.60
Community-GraphRAG (Local)	69.48	62.54	63.01	64.14	62.08	61.66
Community-GraphRAG (Global)	60.76	54.99	54.48	45.72	47.60	45.16
HippoRAG2	67.25	60.42	61.03	**65.31**	**63.26**	**63.01**

Table 2: Performance comparison (%) on the MultiHop-RAG.

Method	Inference	Comparison	Null	Temporal	Overall
RAG	**92.16**	57.59	96.01	30.70	67.02
RaptorRAG	91.91	55.26	90.03	45.28	68.78
KG-GraphRAG (Triplets only)	55.76	22.55	**98.67**	18.70	41.24
KG-GraphRAG (Triplets+Text)	67.40	34.70	97.34	17.15	48.51
Community-GraphRAG (Local)	86.89	60.63	80.07	50.60	69.01
Community-GraphRAG (Global)	89.34	**64.02**	19.27	**53.34**	64.40
HippoRAG2	91.54	58.41	85.71	49.91	**70.27**

4.2 QA Main Results

We first compare the vanilla versions of RAG and GraphRAG variants. Unless otherwise specified, we report main-paper results using Llama-3.1-8B-Instruct. We report results on NQ and HotpotQA, MultiHop-RAG, and NovelQA in Tables 1, 2, and 3, respectively. We summarize key observations below:

(1) **RAG excels on detailed single-hop queries.** RAG achieves strong performance on the single-hop benchmark NQ and on the single-hop (sh) and detail-oriented (dtl) subsets of NovelQA (Tables 1 and 3).

(2) **GraphRAG methods (e.g., HippoRAG2 and Community-GraphRAG (Local)) excel on multi-hop queries.** They perform best on multi-hop QA benchmarks (HotPotQA and MultiHop-RAG) and remain competitive on the multi-hop (mh) subset of NovelQA (Tables 1, 2, and 3).

(3) **Community-GraphRAG (Global) often struggles on QA.** Global search retrieves high-level community summaries, which can lose fine-grained evidence and hurt detail-centric QA, as reflected on detail-oriented subsets in NovelQA. It also performs poorly on Null queries in MultiHop-RAG (ideally answered as `insufficient information`), suggesting increased hallucination risk. However, summary-level retrieval can be beneficial for Comparison and Temporal queries in MultiHop-RAG which require global information.

(4) **KG-based GraphRAG generally underperforms on QA due to limited graph coverage.** KG-GraphRAG retrieves evidence primarily from extracted entities and relations, which can be incomplete and omit answer-critical information. In Appendix A, we show that only about 65.8% of answer entities appear in the constructed KG for HotPotQA and 65.5% for NQ, highlighting the sensitivity of KG-based retrieval to graph construction quality.

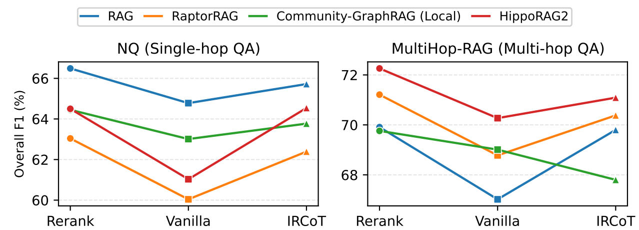


Figure 1: Overall QA performance (F1) under different inference strategies on NQ and MultiHop-RAG.

4.3 QA with Reranking and Iterative Retrieval

In addition to vanilla retrieval, we examine the impact of *reranking* and *iterative retrieval* on QA performance. We conduct experiments on **NQ** and **MultiHop-RAG**, representing single-hop and multi-hop QA scenarios, respectively. Figure 1 summarizes overall performance under different inference strategies.

Across both datasets, reranking and iterative retrieval generally improve performance for all methods compared to vanilla inference, indicating that inference-time enhancements provide gains beyond the underlying retrieval architecture. On **NQ**, both reranking and iterative retrieval (IRCoT) yield consistent improvements, suggesting that such refinements can be beneficial even for predominantly single-hop QA. Importantly, our earlier conclusion remains unchanged: RAG still performs better on single-hop, detail-oriented questions, even when equipped with reranking or iterative retrieval. On the **MultiHop-RAG** benchmark, the gains from inference-time strategies are more pronounced. Both reranking and IRCoT lead to larger absolute improvements than on NQ, highlighting the value of progressive evidence refinement in multi-hop settings. Under these enhanced inference strategies, GraphRAG methods also typically outperform RAG. One exception is Community-GraphRAG (Local) with IRCoT, which exhibits notably low performance on `NULL` queries, despite improvements on other categories. This observation remains consistent with our main findings. Overall, reranking and iterative retrieval are complementary to both RAG and GraphRAG, and are particularly important for multi-hop QA.

4.4 Comparative QA Analysis

In this section, we provide a detailed comparison of RAG and GraphRAG, with a focus on their strengths and weaknesses. We consider vanilla RAG and Community-GraphRAG (Local), and refer to the latter simply as GraphRAG, as it exhibits performance comparable to RAG in our experiments. We partition all queries into four categories: **(1)** queries correctly answered by both methods,

Table 3: Performance comparison (%) on the NovelQA dataset.

RAG	chara	mean	plot	relat	settg	span	times	avg
mh	68.75	52.94	58.33	75.28	92.31	64.00	33.96	47.34
sh	69.08	62.86	66.11	75.00	78.35	-	-	68.73
dtl	64.29	45.51	78.57	10.71	83.78	-	-	55.28
avg	67.78	50.57	67.37	60.80	80.95	64.00	33.96	57.12

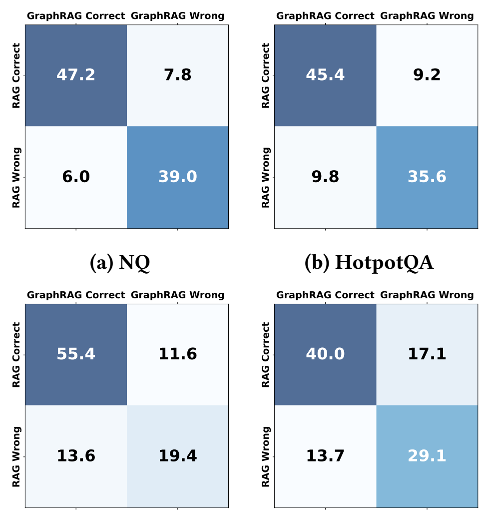
RaptorRAG	chara	mean	plot	relat	settg	span	times	avg
mh	60.42	70.59	63.89	65.17	92.31	52	38.24	48.17
sh	66.45	58.57	65.27	62.50	74.23	-	-	66.25
dtl	62.86	48.88	80.36	28.57	78.38	-	-	57.72
avg	64.44	52.83	67.67	56.80	76.87	52	38.24	57.12

KG-GraphRAG (Triplets+Text)	chara	mean	plot	relat	settg	span	times	avg
mh	52.08	52.94	44.44	55.06	69.23	64.00	28.61	38.37
sh	36.84	45.71	40.17	87.50	36.08	-	-	39.93
dtl	38.57	30.90	42.86	21.43	32.43	-	-	33.60
avg	40.00	36.23	41.09	49.60	38.10	64.00	28.61	37.80

Community-GraphRAG (Local)	chara	mean	plot	relat	settg	span	times	avg
mh	68.75	64.71	55.56	67.42	92.31	52.00	35.83	47.01
sh	59.87	58.57	65.69	87.50	64.95	-	-	63.43
dtl	54.29	37.64	62.50	25.00	70.27	-	-	46.88
avg	60.00	44.91	64.05	59.20	68.71	52.00	35.83	53.03

Community-GraphRAG (Global)	chara	mean	plot	relat	settg	span	times	avg
mh	54.17	58.82	55.56	56.18	53.85	68	20.59	34.39
sh	45.39	50.00	55.65	87.50	38.14	-	-	49.65
dtl	28.57	29.78	32.14	87.50	40.54	-	-	30.89
avg	42.59	36.98	51.66	52.00	40.14	68	20.59	39.17

HippoRAG2	chara	mean	plot	relat	settg	span	times	avg
mh	58.33	64.71	66.67	69.66	92.31	48	37.17	47.84
sh	65.79	65.71	64.44	62.50	72.16	-	-	66.25
dtl	60.00	48.88	69.64	28.57	81.08	-	-	55.83
avg	62.96	54.34	65.56	60.00	76.19	48	37.17	56.54



(a) NQ (b) HotpotQA (c) MultiHop-RAG (d) NovelQA

Figure 2: Confusion matrices comparing GraphRAG and RAG correctness across datasets using Llama 3.1-8B.

(2) queries correctly answered only by RAG (RAG-only), **(3)** queries correctly answered only by GraphRAG (GraphRAG-only), and **(4)** queries incorrectly answered by both methods.

The confusion matrices representing these four groups using the Llama 3.1-8B model are shown in Figure 2. Notably, the proportions of queries correctly answered exclusively by GraphRAG and RAG are significant. For example, 13.6% of queries are GraphRAG-only, while 11.6% are RAG-only on MultiHop-RAG dataset. This phenomenon highlights the complementary properties of RAG and GraphRAG. Therefore, *leveraging their unique advantages has the potential to improve overall performance.*

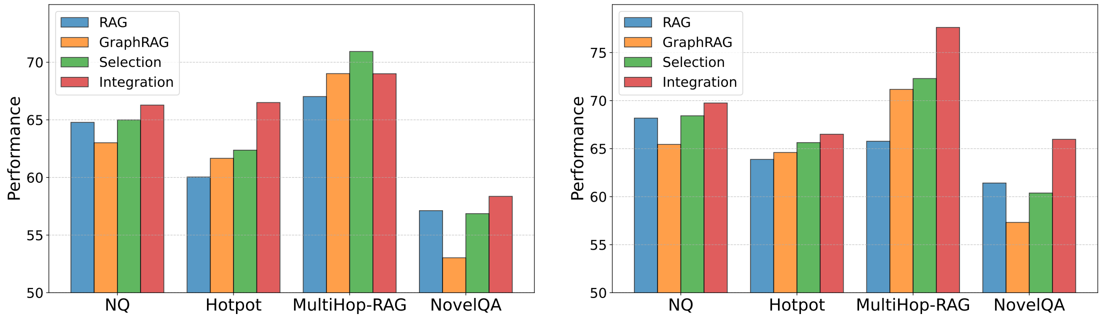
4.5 Improving QA Performance

Building on the complementary properties of RAG and GraphRAG, we investigate the following two strategies to enhance overall QA performance.

Strategy 1: RAG vs. GraphRAG Selection. In Section 4.2, we observe that RAG generally performs well on single-hop queries and those requiring detailed information, while GraphRAG (Community-GraphRAG (Local)) excels in multi-hop queries that require reasoning. Therefore, we hypothesize that RAG is well-suited for fact-based queries, which rely on direct retrieval and detailed information, whereas GraphRAG is more effective for reasoning-based queries that involve chaining multiple facts together. Therefore, given a query, we employ a classification mechanism to determine whether it is fact-based or reasoning-based. Each query is then assigned to either RAG or GraphRAG based on the classification results. Specifically, we leverage the in-context learning ability of LLMs for classification [4, 41]. In this strategy, either RAG or GraphRAG is selected for each query, and we refer to this strategy as **Selection**.

Strategy 2: RAG and GraphRAG Integration. We further explore an **Integration** strategy that jointly leverages the complementary retrieval behaviors of RAG and GraphRAG. For each query, both methods retrieve relevant information in parallel, and the retrieved contexts are concatenated and fed into the generator to produce the final answer. We evaluate the effectiveness of the two proposed strategies on all selected datasets. For MultiHop-RAG and NovelQA, we report overall accuracy, while for NQ and HotPotQA, we use F1 score as the evaluation metric. The results are summarized in Figure 3.

Overall, both strategies consistently improve QA performance across datasets. For instance, on MultiHop-RAG with Llama 3.1-70B, the Selection and Integration strategies improve the best baseline



(a) Llama3.1-8B

(b) Llama3.1-70B

Figure 3: Overall QA performance comparison of different methods.

by 1.1% and 6.4%, respectively. When comparing the two strategies, Integration generally achieves higher performance than Selection. However, Selection processes each query using only a single method, making it more computationally efficient. In contrast, Integration requires running both RAG and GraphRAG for every query, leading to higher computational cost.

4.6 Computation and Storage Analysis

In this subsection, we analyze the computational and storage trade-offs among RAG, KG-GraphRAG, and Community-GraphRAG. Specifically, we report construction time, retrieval latency, and storage footprint on the MultiHop-RAG dataset. The results are summarized in Table 4. From the results, we have the following observations:

- **Construction time:** Both GraphRAG variants incur substantially higher construction cost than RAG, as they require additional graph construction and preprocessing.
- **Retrieval time:** KG-GraphRAG exhibits the highest retrieval latency, primarily due to LLM-based entity expansion and multi-step graph traversal. In contrast, Community-GraphRAG achieves the lowest retrieval latency by relying on direct community-level matching, even outperforming vanilla RAG.
- **Storage:** Community-GraphRAG requires the largest storage footprint, as it stores both community representations and associated summaries. KG-GraphRAG is more storage-efficient than Community-GraphRAG and RAG, reflecting a trade-off between information richness and storage cost.

Table 4: The time and storage analysis on MultiHop-RAG.

Method	Construction Time (s)	Retrieval time (s)	Storage
RAG	135	1724	127MB
KG-GraphRAG	7702	14434	117MB
Community-GraphRAG	5560	1249	165MB

We also report token-matched results across different methods in Appendix C.

4.7 Graph Construction Model

In this subsection, we analyze how graph construction quality affects GraphRAG performance. We focus on Community-GraphRAG (Local) as a representative GraphRAG variant and vary the LLM used for graph construction, while keeping the retrieval and generation components fixed. We evaluate downstream QA performance across multiple tasks and datasets, with detailed results reported in Appendix B. Table 12 summarizes results on MultiHop-RAG across different query categories.

Table 5: Impact of graph construction models on the MultiHop-RAG dataset using Llama 3.1–70B-Instruct.

Graph Construction	Inference	Comparison	NULL	Temporal	Overall
None (RAG)	94.85	56.31	91.36	25.73	65.77
GPT-4o-mini	92.03	60.16	88.70	49.06	71.17
GPT-4o	93.63	66.59	81.06	58.49	75.08

We observe that stronger graph construction models consistently improve QA performance, especially on reasoning-intensive queries such as Comparison and Temporal. While omitting explicit graph construction (i.e., RAG) yields strong performance on Inference and NULL queries, it performs poorly on multi-hop reasoning tasks. In contrast, graphs constructed using more capable LLMs, such as GPT-4o, substantially improve performance on these challenging categories, leading to the highest overall accuracy. These results indicate that GraphRAG performance is sensitive to graph construction quality. However, stronger construction models also incur higher computational cost, highlighting a trade-off between graph quality and system efficiency when selecting LLMs for graph construction.

Summary. Across our QA experiments, we find that RAG and GraphRAG exhibit complementary strengths rather than a clear dominance. RAG consistently performs well on single-hop, fact-centric queries that require precise retrieval of detailed information, while GraphRAG excels on reasoning-intensive, multi-hop queries by explicitly modeling relationships among entities. However, these benefits come with different computational and storage trade-offs, and GraphRAG performance is further influenced by the quality of graph construction. Motivated by these observations, we explore selection and integration strategies that combine the strengths of both paradigms, leading to consistent improvements in overall QA performance. Together, these results suggest that effective QA systems should adaptively balance retrieval precision, reasoning capability, and system efficiency, rather than relying on a single retrieval paradigm.

5 Query-Based Summarization

Query-based summarization is a widely used benchmark for evaluating retrieval-augmented generation (RAG) systems [32, 53]. Recent work has also demonstrated the potential of GraphRAG for summarization tasks [6]. However, Edge et al. [6] focus primarily on global summarization and rely on LLM-as-a-Judge [58] for evaluation. As shown in Section 5.3, LLM-as-a-Judge introduces position bias in summarization evaluation, which can compromise result reliability. Despite the growing interest in GraphRAG, a systematic comparison between RAG and GraphRAG on general query-based summarization tasks across widely used datasets remains unexplored. To address this gap, we conduct a comprehensive evaluation in this section using standard benchmarks and evaluation metrics.

5.1 Datasets and Evaluation Metrics

We evaluate RAG and GraphRAG on four widely used query-based summarization datasets: two single-document datasets, SQuALITY [38] and QMSum [59], and two multi-document datasets, ODSum-story and ODSum-meeting [60]. Unlike the LLM-generated global queries used in the unreleased datasets of Edge et al. [6], most queries in the selected datasets focus on specific roles or events. Since these datasets contain one or more human-written ground truth summaries for each query, we leverage ROUGE-2 [23] and BERTScore [55] as evaluation metrics to measure lexical and semantic similarity between the predicted and ground truth summaries.

5.2 Summarization Experimental Results

We evaluate on vanilla RAG and GraphRAG methods, along with the Integration strategy discussed in Section 4.5. The results of Llama3.1-8B model on Query-based single document summarization and multiple document summarization are shown in Table 6 and Table 7, respectively. Based on these results, we can make the following observations:

(1) **RAG, RaptorRAG, and HippoRAG2 generally performs well on query-based summarization tasks**, primarily because they retrieve original text chunks that are more closely aligned with ground truth.
(2) **KG-based GraphRAG benefit from combining triplets with their corresponding text**. This improves performance by incorporating more details, making predictions closer to the human-written ground truth summaries.
(3) **Community-based GraphRAG performs better with the Local search method**. Local search retrieves entities, relations, and low-level communities, while the Global search method retrieves only high-level summaries. This demonstrates the importance of detailed information in the selected datasets.
(4) **The Integration strategy often performs comparably to RAG alone**, suggesting that simply concatenating RAG and GraphRAG evidence does not reliably improve alignment with detailed ground-truth summaries.

5.3 Position Bias in Existing Evaluation

Based on the results in Section 5.2, Community-based GraphRAG, particularly with global search, generally underperforms RAG on the selected datasets. This observation contrasts with the findings of Edge et al. [6], where Community-based GraphRAG with global search outperformed both local search and RAG.

This discrepancy can be attributed to two key differences between our evaluation and that of Edge et al. [6]. First, their study focuses on global summarization, which aims to capture high-level information from an entire corpus, whereas the datasets used in our evaluation involve queries targeting specific roles or events. Second, Edge et al. [6] evaluate performance using LLM-as-a-Judge without ground-truth references, while we evaluate generated summaries against human-written references using ROUGE and BERTScore. These reference-based metrics emphasize factual coverage and fine-grained details, which may favor different retrieval behaviors.

To further investigate this difference, we follow Edge et al. [6] and evaluate RAG and GraphRAG using LLM-as-a-Judge from two perspectives: *Comprehensiveness* and *Diversity*. Comprehensiveness measures how well a summary covers the details required by the query, while Diversity assesses whether the summary provides a broad and globally inclusive view. For each query, we present summaries generated by RAG and GraphRAG to the LLM and ask it to select the better one under each criterion.

To examine potential position effects in LLM-as-a-Judge evaluations, we consider two presentation orders: Order 1 (O1), where the RAG summary appears first, and Order 2 (O2), where the GraphRAG summary appears first. For each setting, we report the proportion of times each method is preferred by the LLM, where a higher proportion indicates stronger performance under the given presentation order. Figure 4 presents results comparing RAG with GraphRAG (Local) and GraphRAG (Global) on the QMSum and ODSum-story datasets, using GPT-4o as the judge. We make two key observations. First, **position bias [34, 40] is clearly present in LLM-as-a-Judge evaluations for summarization**, as reversing the order of presented summaries leads to substantially different, and in some cases opposite, judgments. This effect is especially pronounced for the comparison between RAG and GraphRAG (Local), as shown in Figures 4a and 4c. Second, in comparisons between RAG and GraphRAG (Global), RAG is consistently preferred in terms of Comprehensiveness, while GraphRAG (Global) is favored for Diversity (Figures 4b and 4d). This suggests that **Community-based GraphRAG with global search emphasizes corpus-level coverage, whereas RAG is more effective at capturing fine-grained, query-specific details**.

We further evaluate stronger LLM judge models, such as GPT-5.4, with results reported in the Appendix. The findings remain consistent. Position bias persists across judge models, although its severity varies: GPT-4o is highly sensitive to presentation order, while GPT-5.4 reduces but does not eliminate this bias. Moreover, the main conclusions remain stable: GraphRAG (Global) improves Diversity, while RAG achieves better Comprehensiveness due to its finer-grained evidence retrieval. We examine graph construction with different LLMs for summarization tasks in Appendix B.

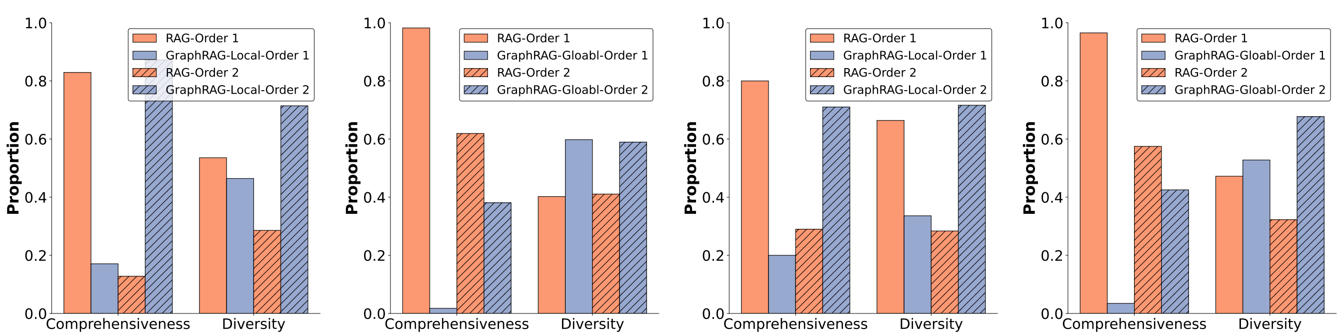
Summary. Across query-based summarization benchmarks, RAG and GraphRAG exhibit different generation characteristic. Under reference-based metrics, RAG typically better matches detailed, query-specific ground-truth summaries, whereas GraphRAG, especially community-based global retrieval, tends to produce more corpus-level and diverse summaries that can deviate from fine-grained details. We also find that LLM-as-a-Judge evaluation can

Table 6: The performance of query-based single document summarization task using Llama 3.1-8B.

Method	SQuALITY						QMSum					
	ROUGE-2			BERTScore			ROUGE-2			BERTScore		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
RAG	15.09	8.74	10.08	74.54	81.00	77.62	21.50	3.80	6.32	81.03	84.45	82.69
RaptorRAG	14.88	8.42	9.81	74.55	81.20	77.71	20.38	**4.17**	**6.68**	**81.64**	84.57	**83.07**
KG-GraphRAG (Triplets only)	11.99	6.16	7.41	82.46	84.30	83.17	13.71	2.55	4.15	80.16	82.96	81.52
KG-GraphRAG (Triplets+Text)	15.00	**9.48**	10.52	**84.37**	**85.88**	**84.92**	16.83	3.32	5.38	80.92	83.64	82.25
Community-GraphRAG (Local)	**15.82**	8.64	10.10	83.93	85.84	84.66	20.54	3.35	5.64	80.63	84.13	82.34
Community-GraphRAG (Global)	10.23	6.21	6.99	82.68	84.26	83.30	10.54	1.97	3.23	79.79	82.47	81.10
HippoRAG2	15.07	8.95	10.20	74.60	81.24	77.75	21.35	4.01	6.60	81.44	**84.63**	83.00
Integration	15.69	9.32	**10.67**	74.56	81.22	77.73	**21.97**	3.80	6.34	80.89	84.47	82.63

Table 7: The performance of query-based multiple document summarization task using Llama3.1-8B.

Method	ODSum-story						ODSum-meeting					
	ROUGE-2			BERTScore			ROUGE-2			BERTScore		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
RAG	15.39	8.44	9.81	**83.87**	**85.74**	84.57	15.50	**6.43**	**8.77**	**83.12**	**85.84**	**84.45**
RaptorRAG	14.69	8.47	9.62	**83.87**	85.76	**84.58**	14.85	6.21	8.44	82.66	85.52	84.06
KG-GraphRAG (Triplets only)	11.02	5.56	6.62	82.09	83.91	82.77	11.64	4.87	6.58	81.13	84.32	82.69
KG-GraphRAG (Triplets+Text)	9.19	5.82	6.22	79.39	83.30	81.03	11.97	4.97	6.72	81.50	84.41	82.92
Community-GraphRAG (Local)	13.84	7.19	8.49	83.19	85.07	83.90	15.65	5.66	8.02	82.44	85.54	83.96
Community-GraphRAG (Global)	9.40	4.47	5.46	81.46	83.54	82.30	11.44	3.89	5.59	81.20	84.50	82.81
HippoRAG2	**15.56**	8.43	**9.82**	83.70	85.71	84.46	**15.91**	6.09	8.51	82.43	85.55	83.95
Integration	14.77	**8.55**	9.53	83.73	85.56	84.40	15.69	6.15	8.51	82.87	85.81	84.31



(a) QMSum Local **(b) QMSum Global** **(c) ODSum-story Local** **(d) ODSum-story Global**

Figure 4: Comparison of LLM-as-a-Judge evaluations for RAG and GraphRAG. "Local" refers to the evaluation of RAG vs. GraphRAG-Local, while "Global" refers to RAG vs. GraphRAG-Global.

be sensitive to presentation order, raising reliability concerns for judge-based comparisons. Overall, these results highlight the need to balance detail fidelity, diversity, evaluation reliability, and system cost when applying (Graph)RAG to query-based summarization.

6 Additional Evaluation and Token Cost Analysis

To further examine whether our findings generalize beyond standard QA and summarization, we evaluate representative methods on two additional settings: LongBench-v2 [1] for long-context reasoning and LoCoMo [28] for long-term memory question answering of LLM agent systems. These benchmarks require models to retrieve, aggregate, and reason over evidence from long documents or multi-session histories. We also provide an end-to-end token cost analysis to better understand the practical trade-offs of different retrieval strategies.

As shown in Table 8, graph-based methods are particularly useful when information must be aggregated across documents or sessions. On LongBench-v2, HippoRAG2 performs best on Multi-Document reasoning, while RAG achieves the highest score on Long In-context Learning. On LoCoMo, HippoRAG2 achieves the best overall performance and improves over RAG on Single-session, Multi-session, and Temporal questions. These results support our

Table 8: Performance of different methods on LongBench-v2 and LoCoMo.

Dataset	Category	RAG	RaptorRAG	HippoRAG2
LongBench-v2	Single-Document	37.5	**41.2**	33.8
LongBench-v2	Multi-Document	35.1	35.1	**40.3**
LongBench-v2	Long In-context Learning	**41.7**	37.5	39.6
LoCoMo	Single-session	0.5955	0.5839	**0.6149**
LoCoMo	Multi-session	0.3362	0.3167	**0.3607**
LoCoMo	Temporal	0.1638	0.1617	**0.1762**
LoCoMo	Open-domain	**0.1084**	0.0983	0.1026
LoCoMo	Adversarial	**0.8004**	0.7964	**0.8004**
LoCoMo	Overall	0.5114	0.5021	**0.5248**

Table 9: End-to-end token usage on LongBench-v2 and LoCoMo.

Dataset	Stage	RAG	RaptorRAG	HippoRAG2
LongBench-v2	Construction-LLM	0	50.2M	184.5M
LongBench-v2	Construction-Embed	32.3M	35.2M	34.0M
LongBench-v2	Retrieval-LLM	0	0	1.5M
LongBench-v2	Generation	622K	521K	624K
LongBench-v2	Total	**32.9M**	85.9M	220.6M
LoCoMo	Construction-LLM	0	287.7K	1.18M
LoCoMo	Construction-Embed	218.4K	239.3K	222.4K
LoCoMo	Retrieval-LLM	0	0	11.30M
LoCoMo	Generation	4.72M	3.65M	4.72M
LoCoMo	Total	4.94M	**4.18M**	17.42M

main conclusion: GraphRAG-style methods are beneficial for distributed information aggregation, while standard RAG remains strong for detail-oriented retrieval and in-context learning.

We further analyze end-to-end token usage across construction, retrieval, and generation. Construction tokens include both LLM tokens used for graph construction or summarization and embedding tokens used for indexing. Retrieval tokens refer to additional LLM calls during retrieval, when applicable. Generation tokens correspond to final answer generation.

Table 9 shows that graph-based methods often require substantially higher token budgets. RaptorRAG introduces additional construction-stage LLM cost for hierarchical summarization, while HippoRAG2 incurs high construction and retrieval-time LLM costs due to graph construction and graph-based reasoning. In contrast, standard RAG mainly uses embedding tokens during indexing and generation tokens during inference. Generation costs are relatively similar across methods under comparable retrieval settings, although RaptorRAG can use fewer generation tokens because its retrieved summaries are typically shorter than raw text chunks.

Overall, these results verify the conclusions from QA and summarization task could be generalized to other tasks.

7 Conclusion

This work presents a unified benchmark evaluation of RAG and GraphRAG across both question answering and query-based summarization, clarifying when explicit graph structures help, and when they do not, under controlled settings. Our analyses reveal strong task-dependent behaviors: RAG is consistently effective for single-hop, detail-oriented queries that require precise evidence, whereas GraphRAG is more advantageous for multi-hop, reasoning-intensive QA and tends to produce more corpus-level, diverse summaries. Motivated by these findings, we study two hybrid strategies, **Selection** and **Integration**, that combine the strengths of both paradigms and improve QA performance. Beyond effectiveness, we highlight practical challenges that limit current GraphRAG systems, including incomplete or noisy graph construction, additional computation and storage overhead, and evaluation artifacts such as position effects in LLM-as-a-Judge for summarization. Together, these observations point toward the next generation of RAG systems: approaches that can construct and refine graphs reliably, adapt retrieval and aggregation to query needs, and deliver stronger reasoning benefits under realistic efficiency constraints.

Acknowledgments

Haoyu Han, Li Ma, Kai Guo, Hui Liu and Jiliang Tang are supported by the National Science Foundation (NSF) under grant numbers CNS2321416, IIS2212032, IIS2212144, IIS 2504089, DUE2234015, CNS2246050, DRL2405483 and IOS2035472, the Michigan Department of Agriculture and Rural Development, US Dept of Commerce, Gates Foundation, Amazon Faculty Award, Meta, NVIDIA, Microsoft and SNAP. Yu Wang, Yongjia Lei and Zhisheng Qi are supported by the National Science Foundation (NSF) under grant number IIS 2524379 and NAIRR 250188.

References

[1] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, et al. 2025. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 3639–3664.

[2] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. 2024. Benchmarking large language models in retrieval-augmented generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 17754–17762.

[3] Jialin Dong, Bahare Fatemi, Bryan Perozzi, Lin F Yang, and Anton Tsitsulin. 2024. Don't Forget to Connect! Improving RAG with Graph-based Reranking. *arXiv preprint arXiv:2405.18414* (2024).

[4] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Tianyu Liu, et al. 2022. A survey on in-context learning. *arXiv preprint arXiv:2301.00234* (2022).

[5] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[6] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).

[7] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2023. Ragas: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217* (2023).

[8] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A survey on rag meeting llms: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6491–6501.

[9] Paulo Finardi, Leonardo Avila, Rodrigo Castaldoni, Pedro Gengo, Celio Larcher, Marcos Piau, Pablo Costa, and Vinicius Caridá. 2024. The Chronicles of RAG: The Retriever, the Chunk and the Generator. *arXiv preprint arXiv:2401.07883* (2024).

[10] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* (2023).

[11] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From rag to memory: Non-parametric continual learning for large language models. *arXiv preprint arXiv:2502.14802* (2025).

[12] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A Rossi, Subhabrata Mukherjee, Xianfeng Tang, et al. 2024. Retrieval-augmented generation with graphs (graphrag). *arXiv preprint arXiv:2501.00309* (2024).

[13] Haoyu Han, Yaochen Xie, Hui Liu, Xianfeng Tang, Sreyashi Nag, William Headden, Yang Li, Chen Luo, Shuiwang Ji, Qi He, et al. 2025. Reasoning with Graphs: Structuring Implicit Knowledge to Enhance LLMs Reasoning. *arXiv preprint arXiv:2501.07845* (2025).
[14] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2023. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232* (2023).
[15] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2023. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research* 24, 251 (2023), 1–43.
[16] Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. *arXiv preprint arXiv:2305.06983* (2023).
[17] Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 59532–59569.
[18] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906* (2020).
[19] Jiho Kim, Sungjin Park, Yeonsu Kwon, Yohan Jo, James Thorne, and Edward Choi. 2023. FactKG: Fact verification via reasoning on knowledge graphs. *arXiv preprint arXiv:2305.06590* (2023).
[20] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics* 7 (2019), 453–466.
[21] Xinze Li, Zhenghao Liu, Chenyan Xiong, Shi Yu, Yu Gu, Zhiyuan Liu, and Ge Yu. 2023. Structure-Aware Language Model Pretraining Improves Dense Retrieval on Structured Data. *arXiv preprint arXiv:2305.19912* (2023).
[22] Yongqi Li, Wenjie Li, and Liqiang Nie. 2022. Dynamic graph reasoning for conversational open-domain question answering. *ACM Transactions on Information Systems (TOIS)* 40, 4 (2022), 1–24.
[23] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.
[24] Fangru Lin, Emanuele La Malfa, Valentin Hofmann, Elle Michelle Yang, Anthony Cohn, and Janet B Pierrehumbert. 2024. Graph-enhanced Large Language Models in Asynchronous Plan Reasoning. *arXiv preprint arXiv:2402.02805* (2024).
[25] Jerry Liu. 2022. *LlamaIndex*. doi:10.5281/zenodo.1234
[26] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. Query rewriting for retrieval-augmented large language models. *arXiv preprint arXiv:2305.14283* (2023).
[27] Yao Ma and Jiliang Tang. 2021. *Deep learning on graphs*. Cambridge University Press.
[28] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 13851–13870.
[29] Fatma Miladi, Valéry Psyché, and Daniel Lemire. 2024. Leveraging GPT-4 for Accuracy in Education: A Comparative Study on Retrieval-Augmented Generation in MOOCs. In *International Conference on Artificial Intelligence in Education*. Springer, 427–434.
[30] Zach Nussbaum, John X Morris, Brandon Duderstadt, and Andriy Mulyar. 2024. Nomic embed: Training a reproducible long context text embedder. *arXiv preprint arXiv:2402.01613* (2024).
[31] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921* (2024).
[32] Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 2023. In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics* 11 (2023), 1316–1331.
[33] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D Manning. 2024. Raptor: Recursive abstractive processing for tree-organized retrieval. In *The Twelfth International Conference on Learning Representations*.
[34] Lin Shi, Chiyu Ma, Wenhua Liang, Weicheng Ma, and Soroush Vosoughi. 2024. Judging the judges: A systematic investigation of position bias in pairwise comparative assessments by llms. *arXiv preprint arXiv:2406.07791* (2024).
[35] Yixuan Tang and Yi Yang. 2024. Multihop-rag: Benchmarking retrieval-augmented generation for multi-hop queries. *arXiv preprint arXiv:2401.15391* (2024).
[36] Yijun Tian, Huan Song, Zichen Wang, Haozhu Wang, Ziqing Hu, Fang Wang, Nitesh V Chawla, and Panpan Xu. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19080–19088.
[37] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. *arXiv preprint arXiv:2212.10509* (2022).
[38] Alex Wang, Richard Yuanzhe Pang, Angelica Chen, Jason Phang, and Samuel R Bowman. 2022. Squality: Building a long-document summarization dataset the hard way. *arXiv preprint arXiv:2205.11465* (2022).
[39] Cunxiang Wang, Ruoxi Ning, Boqi Pan, Tonghui Wu, Qipeng Guo, Cheng Deng, Guangsheng Bao, Qian Wang, and Yue Zhang. 2024. Novelqa: A benchmark for long-range novel question answering. *arXiv preprint arXiv:2403.12766* (2024).
[40] Ziqi Wang, Hanlin Zhang, Xiner Li, Kuan-Hao Huang, Chi Han, Shuiwang Ji, Sham M Kakade, Hao Peng, and Heng Ji. 2024. Eliminating position bias of language models: A mechanistic approach. *arXiv preprint arXiv:2407.01100* (2024).
[41] Jerry Wei, Jason Wei, Yi Tay, Dustin Tran, Albert Webson, Yifeng Lu, Xinyun Chen, Hanxiao Liu, Da Huang, Denny Zhou, et al. 2023. Larger language models do in-context learning differently. *arXiv preprint arXiv:2303.03846* (2023).
[42] Nirmalie Wiratunga, Ramitha Abeyratne, Lasal Jayawardena, Kyle Martin, Stewart Massie, Ikechukwu Nkisi-Orji, Ruvan Weerasinghe, Anne Liret, and Bruno Fleisch. 2024. CBR-RAG: case-based reasoning for retrieval augmented generation in LLMs for legal question answering. In *International Conference on Case-Based Reasoning*. Springer, 445–460.
[43] Yaozu Wu, Yankai Chen, Zhishuai Yin, Weiping Ding, and Irwin King. 2023. A survey on graph embedding techniques for biomedical data: Methods and applications. *Information Fusion* 100 (2023), 101909.
[44] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. 2020. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* 32, 1 (2020), 4–24.
[45] Feng Xia, Ke Sun, Shuo Yu, Abdul Aziz, Liangtian Wan, Shirui Pan, and Huan Liu. 2021. Graph learning: A survey. *IEEE Transactions on Artificial Intelligence* 2, 2 (2021), 109–127.
[46] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv:2309.07597 [cs.CL]
[47] Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2023. Recomp: Improving retrieval-augmented lms with compression and selective augmentation. *arXiv preprint arXiv:2310.04408* (2023).
[48] Ran Xu, Wenqi Shi, Yue Yu, Yuchen Zhuang, Bowen Jin, May D Wang, Joyce C Ho, and Carl Yang. 2024. Ram-ehr: Retrieval augmentation meets clinical predictions on electronic health records. *arXiv preprint arXiv:2403.00815* (2024).
[49] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884* (2024).
[50] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600* (2018).
[51] Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. 2021. QA-GNN: Reasoning with language models and knowledge graphs for question answering. *arXiv preprint arXiv:2104.06378* (2021).
[52] Hao Yu, Aoran Gan, Kai Zhang, Shiwei Tong, Qi Liu, and Zhaofeng Liu. 2024. Evaluation of retrieval-augmented generation: A survey. In *CCF Conference on Big Data*. Springer, 102–120.
[53] Zichun Yu, Chenyan Xiong, Shi Yu, and Zhiyuan Liu. 2023. Augmentation-adapted retriever improves generalization of language models as generic plug-in. *arXiv preprint arXiv:2305.17331* (2023).
[54] Boyu Zhang, Hongyang Yang, Tianyu Zhou, Muhammad Ali Babar, and Xiao-Yang Liu. 2023. Enhancing financial sentiment analysis via retrieval augmented large language models. In *Proceedings of the fourth ACM international conference on AI in finance*. 349–356.
[55] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. 2019. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675* (2019).
[56] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).
[57] Huaixiu Steven Zheng, Swaroop Mishra, Xinyun Chen, Heng-Tze Cheng, Ed H Chi, Quoc V Le, and Denny Zhou. 2023. Take a step back: Evoking reasoning via abstraction in large language models. *arXiv preprint arXiv:2310.06117* (2023).
[58] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems* 36 (2023), 46595–46623.
[59] Ming Zhong, Da Yin, Tao Yu, Ahmad Zaidi, Mutethia Mutuma, Rahul Jha, Ahmed Hassan Awadallah, Asli Celikyilmaz, Yang Liu, Xipeng Qiu, et al. 2021. QMSum: A new benchmark for query-based multi-domain meeting summarization. *arXiv preprint arXiv:2104.05938* (2021).
[60] Yijie Zhou, Kejian Shi, Wencai Zhang, Yixin Liu, Yilun Zhao, and Arman Cohan. 2023. Odsum: New benchmarks for open domain multi-document summarization. *arXiv preprint arXiv:2309.08960* (2023).

Appendix

A Retrieval accuracy of different methods

In this section, we compare the retrieval effectiveness of different methods. Since retrieval does not have explicit ground-truth supervision at the chunk level, we measure *retrieval accuracy* as the proportion of examples for which the ground-truth answer string appears in the retrieved context. We report results on the **HotpotQA** and **NQ** datasets.

Table 10: Retrieval accuracy (%) of different methods on Hotpot and NQ datasets

Method	Hotpot	NQ
RAG	88.60	86.70
KG-GraphRAG (Triplets only)	39.20	32.18
KG-GraphRAG (Triplets+Text)	69.80	61.50
Community-GraphRAG (Local)	67.53	42.20
Community-GraphRAG (Global)	88.60	83.30

As shown in the Table 10, KG-GraphRAG (Triplets only) achieves relatively low retrieval accuracy, particularly on NQ. This is primarily due to the incompleteness of the constructed knowledge graphs—only 65.8% of answer entities exist in the HotpotQA KG, and 65.5% in the NQ KG. In contrast, Community-GraphRAG, which leverages community-level summarization, demonstrates significantly better retrieval performance. These findings highlight several potential directions for improvement:

(1) Enhancing KG construction to increase entity and relation coverage.
(2) Combining structured graph information with raw text to improve retrieval robustness and completeness.

B Graph Construction with different LLMs

In the main paper, we use GPT-4o-mini to extract entities and relationships for graph construction due to cost considerations. To investigate whether stronger LLMs yield better performance, we also use GPT-4o for graph extraction. Specifically, we evaluate this on the MultiHop-RAG and ODSum-story datasets, representing question answering and summarization tasks, respectively. We focused on Community-GraphRAG (Local) as a representative method (GraphRAG) and evaluated it with both LLaMA3.1-8B and LLaMA3.1-70B for generation.

The results are shown in Table 11, Table 12, Table 13 and Table 14, respectively. The results show that using a stronger LLM (GPT-4o) for graph extraction generally improves the performance of GraphRAG on both question answering and summarization tasks. However, the overall conclusion regarding the relative performance of RAG and GraphRAG remains consistent across different graph construction backbones.

Table 11: Performance of different graph construction methods with Llama 3.1–8B on the MultiHop-RAG dataset.

	Inference	Comparison	Null	Temporal	Overall
RAG	92.16	57.59	96.01	30.7	67.02
GPT-4o-mini	86.89	60.63	80.07	50.6	69.01
GPT-4o	88.11	62.62	70.43	49.74	68.74

Table 12: Performance of different graph construction methods with Llama 3.1–70B on the MultiHop-RAG dataset.

70B	Inference	Comparison	Null	Temporal	Overall
RAG	94.85	56.31	91.36	25.73	65.77
GPT-4o-mini	92.03	60.16	88.70	49.06	71.17
GPT-4o	93.63	66.59	81.06	58.49	75.08

Table 13: Performance of different graph construction methods with Llama 3.1–8B on the ODSum-story dataset.

	ROUGE-2			BERTScore		
	P	R	F1	P	R	F1
RAG	15.39	8.44	9.81	83.87	85.74	84.57
GPT-4o-mini	13.84	7.19	8.49	83.19	85.07	83.90
GPT-4o	13.99	7.45	8.64	83.24	85.1	83.94

Table 14: Performance of different graph construction methods with Llama 3.1–8B on the ODSum-meeting dataset.

	ROUGE-2			BERTScore		
	P	R	F1	P	R	F1
RAG	11.85	14.24	11.09	85.96	85.76	85.67
GPT-4o-mini	12.54	10.31	9.61	84.51	85.33	84.71
GPT-4o	12.08	10.84	9.72	84.66	85.28	84.77

C Token-Matched Retrieval Analysis

Beyond runtime and storage, we further analyze whether performance differences are affected by the number of retrieved input tokens. In our default setting, RAG retrieves the top-10 text chunks, while Community-GraphRAG retrieves the top-10 entities together with their associated entity descriptions, relations, relation descriptions, and community summaries. As a result, Community-GraphRAG uses substantially more retrieved tokens than RAG: 9,770 vs. 3,631 tokens on MultiHop-RAG, and 10,244 vs. 2,279 tokens on ODSum-Story.

To control for this difference, we conduct a token-matched experiment by increasing the number of retrieved chunks for RAG so that its input length is comparable to Community-GraphRAG. We report the results using Llama 3.1–70B in Tables 15 and 16.

The results show that increasing RAG's token budget improves performance, especially on MultiHop-RAG. However, the main conclusions remain unchanged. RAG and token-matched RAG remain strong on inference-style queries and summarization tasks, where detailed evidence can be directly retrieved from text chunks. In contrast, GraphRAG remains competitive or better on Comparison and Temporal questions, which require multi-hop reasoning and information aggregation. These results suggest that the differences between RAG and GraphRAG cannot be explained solely by retrieved input length.

Table 15: Performance comparison of RAG, token-matched RAG, and GraphRAG using Llama 3.1–70B on MultiHop-RAG.

Method	Inference	Comparison	Null	Temporal	Overall
RAG	94.85	56.31	91.36	25.73	65.77
RAG-SameToken	95.96	59.58	88.70	43.74	71.01
GraphRAG	92.03	60.16	88.70	49.06	71.17

Table 16: Performance comparison of RAG, token-matched RAG, and GraphRAG using Llama 3.1–70B on ODSum-Meeting.

Method	ROUGE-2			BERTScore		
	P	R	F1	P	R	F1
RAG	11.85	14.24	11.09	85.96	85.76	85.67
RAG-SameToken	12.82	14.07	11.34	85.86	86.00	85.73
GraphRAG	12.54	10.31	9.61	84.51	85.33	84.71

Table 17: Additional LLM-as-a-Judge results for GraphRAG-Local (GR-Local) vs. RAG. We report the winning ratios under two presentation orders, together with the order-averaged GraphRAG-Local win rate.

Dataset	Judge	Metric	O1 (GR-Local:RAG)	O2 (RAG:GR-Local)	Mean GR-Local
QMSum	GPT-4o	Comp	0.97:0.03	0.88:0.12	0.54
QMSum	GPT-4o	Div	0.91:0.09	0.18:0.82	0.87
QMSum	GPT-5.4	Comp	0.44:0.56	0.88:0.12	0.28
QMSum	GPT-5.4	Div	0.62:0.38	0.62:0.38	0.50
ODSum-story	GPT-4o	Comp	0.62:0.38	0.74:0.26	0.44
ODSum-story	GPT-4o	Div	0.63:0.37	0.45:0.55	0.59
ODSum-story	GPT-5.4	Comp	0.47:0.53	0.77:0.23	0.35
ODSum-story	GPT-5.4	Div	0.56:0.44	0.56:0.44	0.50

Table 18: Additional LLM-as-a-Judge results for GraphRAG-Global (GR-Global) vs. RAG. We report the winning ratios under two presentation orders, together with the order-averaged GraphRAG-Global win rate.

Dataset	Judge	Metric	O1 (GR-Global:RAG)	O2 (RAG:GR-Global)	Mean GR-Global
QMSum	GPT-4o	Comp	0.59:0.41	1.00:0.00	0.29
QMSum	GPT-4o	Div	0.91:0.09	0.09:0.91	0.91
QMSum	GPT-5.4	Comp	0.06:0.94	1.00:0.00	0.03
QMSum	GPT-5.4	Div	0.59:0.41	0.44:0.56	0.57
ODSum-story	GPT-4o	Comp	0.26:0.74	0.94:0.06	0.16
ODSum-story	GPT-4o	Div	0.85:0.15	0.26:0.74	0.79
ODSum-story	GPT-5.4	Comp	0.12:0.88	0.97:0.03	0.08
ODSum-story	GPT-5.4	Div	0.61:0.39	0.53:0.47	0.54

D Additional Analysis of Position Bias in LLM-as-a-Judge Evaluation

To assess the position bias, we conduct additional pairwise evaluations between RAG and GraphRAG (GR) on QMSum and ODSum-story. We consider two mitigation strategies: (i) swapping the presentation order and reporting the averaged win rate, and (ii) using multiple judges, including GPT-4o and GPT-5.4, to examine whether the conclusions are robust across judge models.

Tables 17 and 18 report the results for GraphRAG-Local and GraphRAG-Global, respectively. For each setting, we show the winning ratios under two orders, O1 and O2, for Comprehensiveness (Comp) and Diversity (Div). The last column reports the order-averaged GraphRAG win rate, where the GraphRAG score is taken from the corresponding side of each ratio.

We observe that position bias exists for both judges, but its magnitude varies by model. GPT-4o is more order-sensitive, often strongly favoring the first-presented summary. GPT-5.4 reduces this effect but does not fully eliminate it. Despite this bias, the main trends remain stable after order averaging. GraphRAG-Global consistently improves Diversity, confirming that global community-level summaries better capture broad thematic coverage. In contrast, RAG performs better on Comprehensiveness, likely because direct retrieval preserves more fine-grained, query-specific evidence. GraphRAG-Local is generally competitive with RAG, especially on Diversity, where its averaged win rates are close to balanced under GPT-5.4.

These results suggest a simple evaluation protocol for reducing position bias: evaluate both answer orders and report the averaged win rate. When possible, using multiple judges further helps verify the robustness of the conclusions.